CSA08 – PYTHON PROGRAMMING

Assignment Report

Payment Calculator for a Village Milk Co-operative

Item	Details
Student name	Rohit P
Register number	I925260050
Course	CSA08 – Python Programming
Python version	3.12
Bloom's levels	K3 – Apply, K4 – Analyse, K5 – Evaluate
GitHub repository	https://github.com/rohitnew1115-prog/python-CSA0809

Part A: Understanding the Problem

Scenario

The Ksheera Village Milk Producers' Co-operative collects milk twice daily from approximately 300 farmers. Each farmer's entry records the quantity of milk in litres, the fat percentage and the SNF (Solids-Not-Fat) percentage. Farmers are paid once every ten days.

The original payment rules are given below.

Component	Rule
Base rate per litre	Rs. 30.00
Fat adjustment	Plus or minus Rs. 0.40 for every 0.1% above or below 3.5%
SNF adjustment	Plus or minus Rs. 0.30 for every 0.1% above or below 8.5%

The existing calculator has four problems: its totals differ from the bank statement because of calculation errors; some valid entries are rejected; rate revisions are applied only partially; and the rates are maintained in several places, which makes updates difficult. The objective is to develop an accurate, reliable, reusable and scalable payment calculator in Python.

Task 1: Understand the Problem and Identify the Data

K4 – Analyse | 4 Marks

1.1 Analysis of the payment problem

The program must calculate the payment for each farmer from the quantity of milk, the fat percentage and the SNF percentage, and must then total the payments for the ten-day period.

Input	Data type	Description
Farmer ID	String	Unique farmer identification
Litres	Float	Quantity of milk
Fat %	Float	Fat percentage
SNF %	Float	Solids-Not-Fat percentage

The outputs are the adjusted rate per litre, the total amount for each farmer and the ten-day total payment.

1.2 Constants

Constant	Value	Purpose
BASE_RATE	30.00	Basic price per litre
FAT_STANDARD	3.5	Standard fat percentage
FAT_STEP_VALUE	0.40	Adjustment per 0.1% of fat
SNF_STANDARD	8.5	Standard SNF percentage
SNF_STEP_VALUE	0.30	Adjustment per 0.1% of SNF
PAYMENT_DAYS	10	Payment period in days

1.3 Variables

Variable	Meaning
litres	Quantity of milk
fat	Fat percentage
snf	SNF percentage
fat_steps	Number of 0.1% fat adjustments
snf_steps	Number of 0.1% SNF adjustments
rate	Final rate per litre
amount	Payment due to one farmer
total	Total payment for all farmers

1.4 Rules and assumptions

1. Milk quantity must be greater than zero, and negative quantities are not accepted.
2. Fat percentage must lie between 2.0% and 7.0%.
3. SNF percentage must lie within a reasonable range; this program assumes 6.0% to 10.0%.
4. All payment amounts are calculated to two decimal places.
5. The rate is calculated from the original standard values given in the scenario.

1.5 Alternative concept and complexity analysis

Two ways of storing the entries were considered. The first uses three separate lists, one each for litres, fat and SNF. The second uses a single list of tuples, where each tuple holds all the details of one farmer.

```
# Alternative 1: three separate lists
litres = [10, 8, 12]
fat = [3.5, 4.2, 3.1]
snf = [8.5, 8.7, 8.3]

# Alternative 2: list of tuples
entries = [(10, 3.5, 8.5), (8, 4.2, 8.7), (12, 3.1, 8.3)]
```

Comparison	Three lists	List of tuples
Storage	Separate lists	Combined entries
Memory	O(n)	O(n)
Processing	O(n)	O(n)
Chance of indexing error	Higher	Lower
Maintenance	More difficult	Easier

A list of tuples is the better choice because it keeps the related details of one farmer together and reduces indexing mistakes.

1.6 Python program

task01.py

```
BASE_RATE = 30.00
FAT_STANDARD = 3.5
FAT_STEP_VALUE = 0.40
SNF_STANDARD = 8.5
SNF_STEP_VALUE = 0.30

entries = [(10, 3.5, 8.5), (8, 4.2, 8.7), (12, 3.1, 8.3)]

for litres, fat, snf in entries:
    fat_steps = round((fat - FAT_STANDARD) * 10)
    snf_steps = round((snf - SNF_STANDARD) * 10)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    amount = litres * rate
    print(litres, fat, snf, rate, amount)
```

Output

```
10 3.5 8.5 30.0 300.0
8 4.2 8.7 33.4 267.2
12 3.1 8.3 27.799999999999997 333.59999999999997
```

The third entry shows 27.799999999999997 instead of 27.8. The exact values are 27.80 and 333.60, but binary floating-point arithmetic cannot represent them exactly. This is a first example of the precision problem examined in Task 2.

Conclusion: The program identifies the inputs, outputs, constants and rules needed for the payment calculation, and the output already shows why float is unsuitable for money.

Task 2: Explore Values and Types in the Interpreter

K4 – Analyse | 4 Marks

2.1 Data types

```
type(10)
type(10.0)
type("10")
type(True)
type([10, 8, 5])
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>
```

The value 10 is an integer, 10.0 is a float, "10" is a string, True is a Boolean and [10, 8, 5] is a list.

2.2 Division operators

```
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(-7 // 2)
```

Output

```
3.5
3
1
-4
```

The operator / performs true division, // performs floor division and % returns the remainder. Floor division rounds towards negative infinity, so -7 // 2 gives -4 and not -3.

2.3 Floating-point precision

```
print(0.1 + 0.2 == 0.3)
```

Output

```
False
```

Floating-point numbers are stored as binary approximations, and some decimal values such as 0.1 cannot be represented exactly. The sum of 0.1 and 0.2 is therefore slightly different from 0.3.

2.4 Why float should not be used for money

Repeated float calculations accumulate small errors, which can make totals differ from the bank statement. Three alternatives exist. The Decimal type performs exact decimal arithmetic. Integer paise stores money as whole numbers, so Rs. 30.50 becomes 3050. The round() function helps when displaying results but does not remove the underlying representation error.

2.5 Completed code

task02.py

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3


def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total


def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total


print("Float:", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Output

```
Float: 33400.30000000061
```

```
Decimal: 33400.30
False True
```

2.6 Effect of Decimal(float(amount))

```
amount = "0.10"
print(Decimal(amount))
print(Decimal(float(amount)))
```

Output

```
0.10
0.1000000000000000055511151231257827021181583404541015625
```

Creating a Decimal directly from a string preserves the exact decimal value. Converting through float first carries the float's binary approximation into the Decimal, which defeats its purpose.

2.7 Alternative concept and complexity analysis

Method	Memory	Accuracy	Explanation
Decimal	O(1) per value	High	Exact decimal arithmetic
Integer paise	O(1) per value	Exact to the paisa	Rs. 30.50 stored as 3050
Float	O(1) per value	Approximate	Can introduce precision errors

For n payments the time complexity is O(n) and, with a running total, the additional space complexity is O(1).

Conclusion: Decimal is the recommended type for financial calculations because it avoids floating-point precision errors.

Task 3: Choose Variables and Name the Constants

K3 – Apply | 3 Marks

3.1 Naming rules and conventions

- Variable names use lowercase letters, with underscores between words.
- Constant names use uppercase letters.
- Names should describe their purpose.
- Names cannot begin with a digit.
- Reserved keywords cannot be used as names.

```
litres = 10
fat_percentage = 3.5
```

```
snf_percentage = 8.5
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
```

3.2 Why Python has no true constants

Python has no keyword for creating an immutable variable. By convention, an uppercase name such as BASE_RATE signals that the value must not be changed while the program runs, but the language does not enforce this.

3.3 Dynamic typing

```
value = 10
print(type(value))
value = 10.5
print(type(value))
value = "Milk"
print(type(value))
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
```

Python determines the type from the value currently assigned, so the same name can refer to values of different types at different times.

3.4 Multiple assignment

```
litres, fat, snf = 10, 3.5, 8.5
print(litres, fat, snf)
```

Output

```
10 3.5 8.5
```

3.5 Variable swapping

```
a = 10
b = 20
a, b = b, a
print(a, b)
```

Output

```
20 10
```

3.6 Alternative concept and complexity analysis

The first alternative writes the numbers directly in expressions (magic numbers), for example rate = 30 + 4 * 0.40 + 2 * 0.30. A rate revision then requires editing every such expression. The second alternative keeps all rates in one dictionary.

```
RATE_CARD = {
   "base": 30.00,
   "fat": 0.40,
   "snf": 0.30,
}
```

Comparison	Magic numbers	Dictionary
Updating rates	Difficult	Easy
Readability	Low	High
Lookup	Direct expression	O(1) average
Memory	O(1)	O(1) for fixed rates

Conclusion: Named constants or a rate dictionary are better than magic numbers because they improve readability and simplify rate revisions.

Task 4: Write Correct Expressions

K4 – Analyse | 4 Marks

4.1 to 4.4 Core expressions

```
fat_steps = round((fat - 3.5) * 10)
snf_steps = round((snf - 8.5) * 10)
rate = 30.00 + (fat_steps * 0.40) + (snf_steps * 0.30)
amount = litres * rate
total = sum(payments)
```

4.5 Operator precedence

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

Output

```
14
20
```

Multiplication is performed before addition unless parentheses change the order.

4.6 Why 4.2 - 3.5 can give an unexpected result

```
print(4.2 - 3.5)
```

Output

```
0.7000000000000002
```

Neither 4.2 nor 3.5 is stored exactly in binary, so the difference carries a small error. Multiplying by 10 and rounding recovers the intended whole number of 0.1% steps.

4.7 round(), int() and math.floor()

Function	Example	Result
round()	round(4.2)	4
int()	int(4.8)	4
math.floor()	math.floor(-4.2)	-5

round() gives the nearest integer, with ties going to the nearest even number. int() discards the fractional part, so it truncates towards zero. math.floor() rounds towards negative infinity.

4.8 Comparison chaining

```
fat = 4.2
print(2.0 <= fat <= 7.0)
```

Output

```
True
```

Python evaluates both comparisons together, which gives a compact range check.

4.9 Unexpected expressions

```
print(10 + 5 * 2)
print(10 / 4)
print(10 // 4)
print(-7 // 2)
```

Output

```
20
2.5
```

```
2
-4
```

4.10 Alternative concept and complexity analysis

An alternative is to use int() or math.floor() instead of round() when counting steps. This fails because the floating-point error can leave the product just below the intended integer. For fat = 4.1 the intended number of steps is 6, but:

```
print((4.1 - 3.5) * 10)
print(int((4.1 - 3.5) * 10))
print(round((4.1 - 3.5) * 10))
```

Output

```
5.9999999999999964
5
6
```

Truncation therefore pays one step too few, while round() gives the correct count. The time and space complexity per entry are both O(1).

Conclusion: Correct expressions, careful use of parentheses and rounding to the nearest step are necessary for an accurate rate.

Task 5: Read Input and Convert Types

K3 – Apply | 3 Marks

5.1 Reading the clerk's entries

The input() function always returns a string, so values must be converted before they are used in calculations.

```
litres = input("Enter litres: ")
print(type(litres))
```

Output

```
<class 'str'>
```

5.2 Converting the input

```
litres = float(input("Enter litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
```

5.3 Validating the input

task05.py

```
def read_entry():
    try:
        litres = float(input("Enter litres: "))
        fat = float(input("Enter fat %: "))
        snf = float(input("Enter SNF %: "))
    except ValueError:
        print("Please enter valid numbers")
        return None

    if litres <= 0:
        print("Invalid quantity")
        return None
    if not 2.0 <= fat <= 7.0:
        print("Invalid fat percentage")
        return None
    if not 6.0 <= snf <= 10.0:
        print("Invalid SNF percentage")
        return None
    return litres, fat, snf



entry = read_entry()
print("Entry:", entry)
```

5.4 Handling invalid entries

Input	Expected result
Litres = 10	Valid quantity
Litres = -5	Invalid quantity
Litres = 8.5	Valid numeric input
Litres = eight	Rejected: not a number
Fat = 8.0	Invalid fat percentage

5.5 Formatting output with f-strings

```
amount = 334.567
print(f"Payment: Rs. {amount:.2f}")
```

Output

```
Payment: Rs. 334.57
```

5.6 Alternative concept and complexity analysis

An alternative is to inspect the input string before converting it. The approach used here converts directly with float() inside a try-except block and then checks the ranges.

Method	Advantage	Disadvantage
String checking	Can check the input format	More code is required
Try-except	Handles every conversion error	Needs exception handling

Conclusion: Converting with float() inside try-except, followed by range validation, ensures that only valid milk entries are accepted.

Task 6: Organise the Logic into Functions

K3 – Apply | 4 Marks

6.1 Why functions are required

Functions divide a large program into smaller, reusable sections. Three functions are required: rate_per_litre() calculates the rate, payout() calculates one farmer's payment and ten_day_total() calculates the total for all farmers.

6.2 Complete program

task06.py

```
BASE_RATE = 30.00
FAT_STANDARD = 3.5
FAT_STEP_VALUE = 0.40
SNF_STANDARD = 8.5
SNF_STEP_VALUE = 0.30



def rate_per_litre(fat, snf):
    """Return the milk rate per litre."""
    fat_steps = round((fat - FAT_STANDARD) * 10)
    snf_steps = round((snf - SNF_STANDARD) * 10)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE
    rate += snf_steps * SNF_STEP_VALUE
    return round(rate, 2)



def payout(litres, fat, snf):
    """Return the payment for one farmer."""
    return round(litres * rate_per_litre(fat, snf), 2)



def ten_day_total(entries):
    """Return the total payment for all entries."""
    total = 0
    for litres, fat, snf in entries:
```

```
        total += payout(litres, fat, snf)
    return round(total, 2)


entries = [(10, 3.5, 8.5), (8, 4.2, 8.7), (12, 3.1, 8.3)]
for litres, fat, snf in entries:
    print("Payment:", payout(litres, fat, snf))
print("Ten-day total:", ten_day_total(entries))
```

Output

```
Payment: 300.0
Payment: 267.2
Payment: 333.6
Ten-day total: 900.8
```

6.3 Local and global variables

A global variable is defined outside any function and is visible throughout the program. A local variable is defined inside a function and cannot normally be accessed outside it.

```
BASE_RATE = 30.00


def calculate():
    litres = 10
    return litres * BASE_RATE


print(calculate())
```

Output

```
300.0
```

6.4 Returning versus printing

return	print
Sends a value back to the caller	Displays a value on the screen
The value can be reused	Used mainly for presentation
Suitable for calculations	Suitable for output

6.5 Alternative concept and complexity analysis

Without functions, all calculations would sit in one long script. With functions, duplicated code is reduced, debugging is easier and each calculation can be tested individually. For n entries

the time complexity is O(n) and the additional space is O(1) for the running total, excluding the stored entries.

Conclusion: Functions improve modularity, readability, reusability and maintenance.

Task 7: Build a Module

K4 – Analyse | 4 Marks

7.1 The module milkrate.py

milkrate.py

```
"""Rate card of the village milk co-operative."""

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30



def rate_per_litre(fat, snf):
    """Return the rate per litre rounded to two decimals."""
    fat_steps = round((fat - 3.5) * 10)
    snf_steps = round((snf - 8.5) * 10)
    rate = BASE_RATE
    rate += fat_steps * FAT_STEP_VALUE
    rate += snf_steps * SNF_STEP_VALUE
    return round(rate, 2)



def payout(litres, fat, snf):
    """Return the farmer's payment rounded to two decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)



if __name__ == "__main__":
    entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]
    for litres, fat, snf in entries:
        rate = rate_per_litre(fat, snf)
        print(litres, fat, snf, rate, payout(litres, fat, snf))
```

7.2 The script main.py

main.py

```
import milkrate

entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]

for litres, fat, snf in entries:
    rate = milkrate.rate_per_litre(fat, snf)
    amount = milkrate.payout(litres, fat, snf)
    print(litres, fat, snf, rate, amount)
```

7.3 Output of main.py

Output

```
10 3.5 8.5 30.0 300.0
8.5 4.2 8.7 33.4 283.9
12 3.1 8.3 27.8 333.6
```

7.4 Difference between the import methods

```
import milkrate
print(milkrate.rate_per_litre(4.2, 8.7))

from milkrate import payout
print(payout(10, 3.5, 8.5))
```

The statement import milkrate loads the whole module, and its functions are called with the module name as a prefix. The statement from milkrate import payout loads only the named function, which is then called directly.

7.5 Purpose of if __name__ == "__main__"

This condition runs the test code only when the file is executed directly. When another program imports the module, the test code is skipped. Without the condition, the test statements would also run on every import.

7.6 Standard modules

Python provides standard modules such as math for mathematical operations, decimal for exact decimal arithmetic and datetime for dates and times.

7.7 Alternative concept and complexity analysis

Comparison	Copying functions	Using a module
Code duplication	High	Low
Updating rates	Several files	One file
Maintenance	Difficult	Easy
Import overhead	None	One-time loading

The rate and payment calculations are O(1) each, the total for n farmers is O(n), and the space is O(n) if all entries are stored.

Conclusion: A separate module improves reuse, removes duplication and makes future rate changes easier.

Task 8: Plan the Algorithm Before Coding

K3 – Apply | 4 Marks

8.1 Steps of algorithmic problem solving

1. Understand: identify the inputs, outputs and payment rules.
2. Plan: write the algorithm and pseudocode.
3. Write: convert the pseudocode into Python.
4. Test: check normal, boundary and invalid inputs.

8.2 Pseudocode for one farmer

```
ALGORITHM MILK_PAYMENT
START
  INPUT litres, fat, snf
  IF litres <= 0 THEN
     OUTPUT "Invalid quantity"
  ELSE IF fat < 2.0 OR fat > 7.0 THEN
     OUTPUT "Invalid fat percentage"
  ELSE IF snf < 6.0 OR snf > 10.0 THEN
     OUTPUT "Invalid SNF percentage"
  ELSE
     SET fat_steps = ROUND((fat - 3.5) * 10)
     SET snf_steps = ROUND((snf - 8.5) * 10)
     SET rate = 30.00 + fat_steps * 0.40 + snf_steps * 0.30
     SET amount = litres * rate
     OUTPUT rate, amount
  ENDIF
END
```

8.3 Pseudocode for the ten-day total

```
ALGORITHM TEN_DAY_TOTAL
START
  SET total TO 0
  FOR each entry IN entries
     IF entry is valid THEN
        SET total TO total + payout(entry)
     ELSE
        OUTPUT "Invalid entry"
     ENDIF
  ENDFOR
  OUTPUT total
END
```

8.4 Flowchart

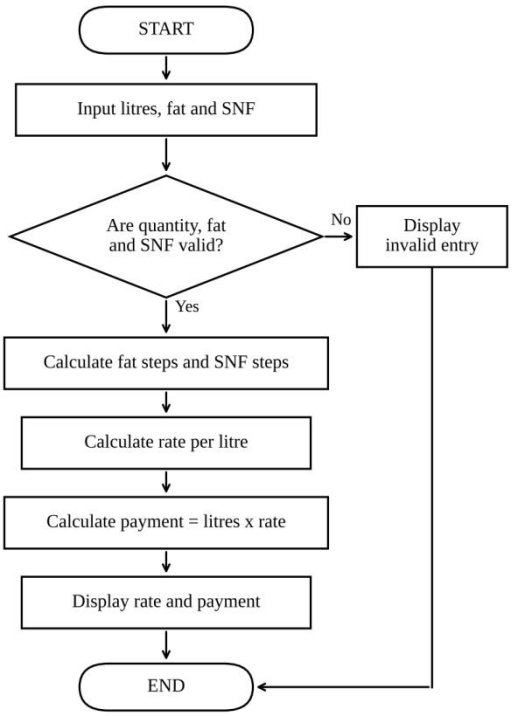


Figure 1: Flowchart for the payment of one farmer

8.5 Hand trace for three entries

Entry	Litres	Fat	SNF	Rate (Rs.)	Payment (Rs.)
1	10	3.5	8.5	30.00	300.00
2	8	4.2	8.7	33.40	267.20
3	12	3.1	8.3	27.80	333.60
Total	30				900.80

8.6 Alternative concept and complexity analysis

```
# Alternative 1: running total
total = 0
for entry in entries:
    total += payout(*entry)

# Alternative 2: store every payment first
payments = []
for entry in entries:
    payments.append(payout(*entry))
total = sum(payments)
```

Comparison	Running total	Store all payments
Time	O(n)	O(n)
Extra space	O(1)	O(n)
Keeps individual payments	No	Yes
Memory requirement	Low	Higher

Conclusion: A running total is preferable when only the final amount is required. Storing the payments is useful when individual records must be kept.

Task 9: Implement and Test the Calculator

K3 – Apply | 4 Marks

9.1 Complete program

The program uses Decimal arithmetic, validates every entry and rounds each payment to the nearest paisa using ROUND_HALF_UP.

calculator.py

```
from decimal import Decimal, ROUND_HALF_UP
BASE_RATE=Decimal("30.00"); FAT_STANDARD=Decimal("3.5"); SNF_STANDARD=Decimal("8.5")
FAT_STEP=Decimal("0.40"); SNF_STEP=Decimal("0.30")
def calculate_rate(fat,snf):
    fs=((fat-FAT_STANDARD)*10).quantize(Decimal("1"),rounding=ROUND_HALF_UP)
    ss=((snf-SNF_STANDARD)*10).quantize(Decimal("1"),rounding=ROUND_HALF_UP)
    return (BASE_RATE+fs*FAT_STEP+ss*SNF_STEP).quantize(Decimal("0.01"))
def validate_entry(litres,fat,snf):
    if litres<=0: raise ValueError("Quantity must be positive")
    if not Decimal("2.0")<=fat<=Decimal("7.0"): raise ValueError("Invalid fat percentage")
    if not Decimal("6.0")<=snf<=Decimal("10.0"): raise ValueError("Invalid SNF percentage")
def calculate_payment(litres,fat,snf):
    validate_entry(litres,fat,snf)
    r=calculate_rate(fat,snf)
    return r,(litres*r).quantize(Decimal("0.01"),rounding=ROUND_HALF_UP)
def ten_day_total(entries):
    t=Decimal("0.00")
    for l,f,s in entries: t+=calculate_payment(l,f,s)[1]
    return t
```

9.2 Test cases

Test	Litres	Fat	SNF	Expected result
Normal 1	10	3.5	8.5	Rs. 300.00
Normal 2	8	4.2	8.7	Rs. 267.20
Normal 3	12	3.1	8.3	Rs. 333.60
Upper boundary	10	7.0	10.0	Rs. 485.00

Test	Litres	Fat	SNF	Expected result
Lower boundary	10	2.0	6.0	Rs. 165.00
Invalid quantity	-5	3.5	8.5	Rejected
Invalid fat	10	8.0	8.5	Rejected
Invalid SNF	10	3.5	11.0	Rejected
Invalid text	abc	3.5	8.5	Rejected

9.3 Manual calculation of the upper boundary test

Litres = 10, fat = 7.0%, SNF = 10.0%.

```
Fat steps = (7.0 - 3.5) x 10 = 35
SNF steps = (10.0 - 8.5) x 10 = 15
Rate = 30 + (35 x 0.40) + (15 x 0.30) = 30 + 14 + 4.50 = Rs. 48.50
Payment = 10 x 48.50 = Rs. 485.00
```

The lower boundary is checked in the same way. Fat steps = -15 and SNF steps = -25, so the rate is 30 - 6.00 - 7.50 = Rs. 16.50 and the payment for 10 litres is Rs. 165.00.

9.4 Automated testing and results

Python assert statements allow all test cases to be repeated automatically after every change.

```
D = Decimal
assert calculate_rate(D("3.5"), D("8.5")) == D("30.00")
assert calculate_payment(D("10"), D("7.0"), D("10.0")) == (D("48.50"), D("485.00"))
assert ten_day_total([(D("10"), D("3.5"), D("8.5")),
                      (D("8"), D("4.2"), D("8.7")),
                      (D("12"), D("3.1"), D("8.3"))]) == D("900.80")
print("All tests passed")
```

Running the complete test script produced the following results.

Output

```
10 3.5 8.5 30.00 300.00
8 4.2 8.7 33.40 267.20
12 3.1 8.3 27.80 333.60
Total 900.80
(Decimal('48.50'), Decimal('485.00'))
Rejected: Quantity must be positive
Rejected: Invalid fat percentage
Rejected: Invalid SNF percentage
Rejected: abc -> InvalidOperation
All tests passed
```

Automated testing saves time, removes repeated manual work and detects errors introduced by later changes. Each test is O(1), n tests take O(n) time, and the additional space is O(1) excluding the test data.

Conclusion: The calculator passes all normal, boundary and invalid test cases, and automated tests make repeated verification efficient.

Task 10: Compare Expectations with Execution

K4 – Analyse | 4 Marks

10.1 Expected and actual results

Farmer	Expected payment (Rs.)	Actual payment (Rs.)
1	300.00	300.00
2	267.20	267.20
3	333.60	333.60
Total	900.80	900.80

The Decimal-based implementation matches the expected calculation exactly.

10.2 Float and Decimal compared

The Task 2 totals were 33400.30000000061 using float and 33400.30 using Decimal. The difference is 33400.30000000061 - 33400.30 = 0.00000000061, that is about 6.1 x 10^-10 rupees. The amount is tiny for one total, but such errors accumulate over many entries and make exact reconciliation with the bank statement unreliable.

10.3 Effect of rounding each payment

Consider three identical payments of Rs. 10.005 each.

```
from decimal import Decimal, ROUND_HALF_UP

each = Decimal("10.005").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
print(each * 3)
print((Decimal("10.005") * 3).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))
```

Output

```
30.03
30.02
```

Rounding each payment first gives 30.03, while rounding only the final sum gives 30.02. The co-operative must therefore adopt one consistent rule. The recommended rule is to calculate each farmer's payment in Decimal and round it to the nearest paisa before it is added to the ten-

day total, because each farmer is paid exactly the rounded amount shown on his or her statement.

10.4 Time measurement

task10.py

```
from decimal import Decimal
from time import perf_counter

start = perf_counter()
total = Decimal("0.00")
for i in range(100000):
    total += Decimal("30.00")
end = perf_counter()

print("Total:", total)
print("Execution time:", end - start)
```

Output

```
Total: 3000000.00
Execution time: 0.0205 (seconds; this value varies from machine to machine)
```

The timing depends on the computer, the Python version and the system load, so the figure should be re-measured on the machine used for submission. It shows that 100,000 Decimal additions complete in a small fraction of a second, so Decimal is fast enough for about 300 farmers.

10.5 Alternative concept and complexity analysis

An alternative is to store previously calculated rates in a dictionary keyed by the fat and SNF values multiplied by ten.

```
rate_table = {
    (35, 85): Decimal("30.00"),
    (42, 87): Decimal("33.40"),
    (31, 83): Decimal("27.80"),
}
```

Method	Processing	Complexity
Calculate every time	Repeats the rate calculation	O(1) per entry
Dictionary lookup	Reuses stored rates	O(1) average per entry
Building the table	One-time cost for k combinations	O(k) time and space

Conclusion: Both approaches are suitable. A rate table helps when many farmers share the same readings, but it needs extra memory and must be rebuilt whenever rates change.

Task 11: Recommend and Defend the Final Design

K5 – Evaluate | 4 Marks

11.1 Recommended design

A modular, Decimal-based payment calculator with named constants, input validation and automated testing is recommended.

11.2 Recommended data types

Data	Type	Reason
Farmer ID	String	Supports letters and digits
Litres	Decimal	Exact measurements
Fat and SNF	Decimal	Exact percentage readings
Rate	Decimal	Financial accuracy
Payment	Decimal	Avoids float precision errors
Farmer records	List of dictionaries	Organised, readable storage

11.3 Recommended rounding rule

Payments are rounded to two decimal places using ROUND_HALF_UP, which matches the usual accounting convention.

```
from decimal import Decimal, ROUND_HALF_UP

amount = Decimal("300.567")
print(amount.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))
```

Output

```
300.57
```

11.4 Carrying out a rate revision

All rate values are kept in one rate card. When the co-operative announces a new rate, only the rate card is edited.

```
RATE_CARD = {
    "base": Decimal("30.00"),
    "fat": Decimal("0.40"),
    "snf": Decimal("0.30"),
}
```

11.5 Limitations of the design

- Incorrect analyser readings produce incorrect payments.

- Rates need to be updated whenever a revision is announced.
- Data must be backed up to prevent loss.
- Secure access requires user authentication.
- Large-scale use requires database storage and audit records.

11.6 Suitability for district-union scale

The modular design can be extended to a district union. It can support multiple collection centres, thousands of farmer records, centralised rate management, database storage, payment reports and audit logs.

11.7 Alternative concept: external JSON rate card

Rates can be held in a JSON file instead of in the Python source.

rates.json

```
{
  "base": "30.00",
  "fat": "0.40",
  "snf": "0.30"
}
```

final_design.py

```
import json
from decimal import Decimal

with open("rates.json", "r") as file:
  RATE_CARD = json.load(file)

BASE_RATE = Decimal(RATE_CARD["base"])
FAT_STEP = Decimal(RATE_CARD["fat"])
SNF_STEP = Decimal(RATE_CARD["snf"])
```

Method	Advantage	Disadvantage
Rates in Python	Simple and direct	Code must be edited
JSON file	Rates updated without code changes	File needs validation
Database	Centralised management	More set-up required

Conclusion: For the initial system, a modular Python program using Decimal and a central rate card is the best option. JSON or database storage can be introduced as the system grows.

Part B: Day-4 Design Challenge

Task 12: Re-evaluate and Adapt the Design

K5 – Evaluate | 8 Marks

12.1 Impact analysis

The automatic milk analyser now reports fat and SNF with two decimal places, and the co-operative has introduced seasonal rates and a bulk bonus.

Particulars	April to June	Other months
Base rate	Rs. 32.00	Rs. 30.00
Fat adjustment (per 0.1%)	Rs. 0.45	Rs. 0.40
SNF adjustment (per 0.1%)	Rs. 0.30	Rs. 0.30
Bonus	Rs. 0.50 per litre	Rs. 0.50 per litre

The bonus is paid on all litres supplied by a farmer in a ten-day period if that total is more than 500 litres. A farmer supplying exactly 500 litres receives no bonus.

What has changed

1. The analyser provides more precise readings.
2. The base rate depends on the season.
3. The fat adjustment changes during April to June.
4. A new bonus is introduced.
5. The bonus depends on the total quantity supplied in ten days.

Constants affected

BASE_RATE, FAT_STEP_VALUE and SNF_STEP_VALUE now vary by season, and two new constants, BONUS_RATE and BONUS_LIMIT, are required.

Functions affected

rate_per_litre() must select the correct seasonal rate card. payout() must include the bonus. ten_day_total() must check the total litres supplied by each farmer.

Effect of two-decimal readings on rounding

With two-decimal readings the step calculation can produce a half step. For fat = 4.25%, (4.25 - 3.50) x 10 = 7.5. Using ROUND_HALF_UP gives 8 steps. The calculation must not truncate with int(), which would give 7. Note that ROUND_HALF_UP rounds halves away from zero, so a half step below the standard, such as -7.5, becomes -8.

12.2 Evaluation of the original design

The modular design remains appropriate but needs modification.

Existing design	New requirement	Decision
Fixed base rate	Seasonal base rate	Modify
Fixed fat adjustment	Seasonal fat adjustment	Modify
SNF adjustment	Unchanged	Retain
No bonus	Quantity-based bonus	Add
Decimal arithmetic and rounding	Two-decimal readings	Retain
Separate functions	Reusable functions	Retain

Worked example 1: April to June

Litres = 100, fat = 4.25%, SNF = 8.65%.

```
Fat steps = (4.25 - 3.50) x 10 = 7.5, rounded to 8
SNF steps = (8.65 - 8.50) x 10 = 1.5, rounded to 2
Rate = 32 + (8 x 0.45) + (2 x 0.30) = 32 + 3.60 + 0.60 = Rs. 36.20
Payment = 100 x 36.20 = Rs. 3,620.00
```

Worked example 2: other months

```
Rate = 30 + (8 x 0.40) + (2 x 0.30) = 30 + 3.20 + 0.60 = Rs. 33.80
Payment = 100 x 33.80 = Rs. 3,380.00
```

Worked example 3: bonus applies

A farmer supplies 550 litres in April to June at Rs. 36.20 per litre.

```
Bonus = 550 x 0.50 = Rs. 275.00
Payment = (550 x 36.20) + 275 = 19,910 + 275 = Rs. 20,185.00
```

Worked example 4: exactly 500 litres

The bonus requires more than 500 litres, and 500 is not greater than 500, so the bonus is zero.

```
Payment = 500 x 36.20 = Rs. 18,100.00
```

12.3 Adaptation of the design

The revised program keeps both rate cards in a dictionary, selects the card from the month, rounds steps with ROUND_HALF_UP and applies the bonus to the farmer's ten-day litre total. It assumes that the quality readings used for the ten-day payment are a single representative fat and SNF pair for that farmer.

seasonal_calculator.py

```
from decimal import Decimal, ROUND_HALF_UP
```

```
BONUS_RATE = Decimal("0.50")
BONUS_LIMIT = Decimal("500")

RATE_CARDS = {
   "summer": {"base": Decimal("32.00"), "fat": Decimal("0.45"),
            "snf": Decimal("0.30")},
   "normal": {"base": Decimal("30.00"), "fat": Decimal("0.40"),
            "snf": Decimal("0.30")},
}


def get_season(month):
   """Select the rate card for the given month (1 to 12)."""
   return "summer" if 4 <= month <= 6 else "normal"


def steps(value, standard):
   """Number of 0.1% steps from the standard, rounded half up."""
   return ((value - standard) * 10).quantize(Decimal("1"),
                                   rounding=ROUND_HALF_UP)


def calculate_rate(fat, snf, season):
   """Return the rate per litre for the selected season."""
   card = RATE_CARDS[season]
   rate = (card["base"]
           + steps(fat, Decimal("3.50")) * card["fat"]
           + steps(snf, Decimal("8.50")) * card["snf"])
   return rate.quantize(Decimal("0.01"))


def ten_day_payment(total_litres, fat, snf, month):
   """Return season, rate, bonus and total payment for ten days."""
   season = get_season(month)
   rate = calculate_rate(fat, snf, season)
   bonus = Decimal("0")
   if total_litres > BONUS_LIMIT:
      bonus = total_litres * BONUS_RATE
   total = (total_litres * rate + bonus).quantize(
      Decimal("0.01"), rounding=ROUND_HALF_UP)
   return season, rate, bonus, total
```

The program was run for both seasons and for litre totals on either side of the bonus limit, using fat = 4.25% and SNF = 8.65%.

Test	Month	Litres	Rate (Rs.)	Bonus (Rs.)	Total payment (Rs.)
Summer, above limit	5	550	36.20	275.00	20,185.00
Summer, at limit	5	500	36.20	0.00	18,100.00
Other season, above limit	8	550	33.80	275.00	18,865.00

Test	Month	Litres	Rate (Rs.)	Bonus (Rs.)	Total payment (Rs.)
Other season, below limit	8	400	33.80	0.00	13,520.00

These results agree with worked examples 1 to 4. The program also reproduces the single-reading rates of Rs. 36.20 and Rs. 33.80.

Why this redesign is better

- Seasonal rates are held in one dictionary.
- The same calculation function serves both seasons.
- Decimal arithmetic keeps the financial results exact.
- The bonus is tested against the farmer's ten-day total.
- Policy changes need edits to the data and not a rewrite of the program.

12.4 Evidence-based defence

Feature	Original design	Revised design
Base rate	Fixed Rs. 30	Seasonal: Rs. 32 or Rs. 30
Fat adjustment	Rs. 0.40	Seasonal: Rs. 0.45 or Rs. 0.40
SNF adjustment	Rs. 0.30	Rs. 0.30 (retained)
Analyser reading	One decimal place	Two decimal places
Bonus	Not available	Rs. 0.50 per litre
Bonus condition	Not applicable	More than 500 litres in ten days
Structure	Modular	Modular with seasonal rate cards

The revised design supports the automatic analyser, handles seasonal revisions, calculates the bonus correctly, maintains financial accuracy, reduces code duplication and can be extended to more collection centres. Its limitations are that the month must be supplied correctly, analyser readings must be valid, farmer records must be grouped correctly for the ten-day bonus, and the rate card must be updated whenever policy changes.

Final recommendation: Modify the original design rather than replacing it. Its modular structure is sound, so only seasonal rate selection and the ten-day bonus need to be added, and Decimal arithmetic should be retained.

12.5 Complexity analysis

Operation	Time	Space
Select season	O(1)	O(1)
Calculate rate	O(1)	O(1)
Calculate one farmer's payment	O(1)	O(1)

Operation	Time	Space
Total litres and payments for n entries	O(n)	O(1) additional
Store the n entries	O(n)	O(n)

The revised design does not change the overall time complexity, which remains O(n).

Final conclusion: The modified calculator is more flexible and accurate than the original, and it suits the co-operative's new requirements.

Part C: GitHub and Assignment Submission

1. Folder structure

Folder	Contents
Task01 to Task06	task01.py to task06.py
Task07	milkrate.py, main.py
Task08	pseudocode.txt and the flowchart image
Task09	calculator.py
Task10	performance.py
Task11	final_design.py, rates.json
Task12	seasonal_calculator.py
screenshots	Execution screenshots for each task
README.md	Project description and instructions

2. README.md content

```
Payment Calculator for a Village Milk Co-operative
Course: CSA08 - Python Programming

Student Details
Name:
Register Number:
Python Version: 3.12

Project Description
A Python milk payment calculator for a village milk producers'
co-operative. It calculates each farmer's payment from the quantity
of milk, fat percentage and SNF percentage.

Features
- Milk payment calculation with fat and SNF adjustments
- Input validation
- Decimal-based financial calculations
- Modular programming
- Seasonal rate revisions and ten-day bonus
```

```
- Automated testing

Technologies Used
Python 3.12, decimal module, json module, GitHub

How to Run
1. Install Python.
2. Download or clone this repository.
3. Open the project folder.
4. Run the required file, for example:
   python Task12/seasonal_calculator.py
```

3. Evidence to collect

Task	Required evidence
1	Data analysis and variables
2	Interactive Python session
3	Constants and naming conventions
4	Expression outputs
5	Input conversion and validation
6	Function execution
7	Module import demonstration
8	Pseudocode, flowchart and hand trace
9	Program execution and test results
10	Expected versus actual comparison
11	Final design explanation
12	Seasonal calculation and bonus tests

Screenshots must show the actual execution of each program on the submitting machine.

4. Final submission checklist

1. Complete all 12 tasks.
2. Execute every Python program and correct any errors.
3. Capture screenshots of the actual output.
4. Include the name and register number in the README.
5. Include the Python version and library information.
6. Upload all source files to GitHub and commit the changes.
7. Verify that the repository is accessible.
8. Add the GitHub link to the report and the assignment portal.

Overall Conclusion

The Payment Calculator for a Village Milk Co-operative applies variables, constants, data types, operators, input validation, functions, modules, algorithms and testing. The final system uses Decimal arithmetic, seasonal rate cards and a ten-day bonus to improve the accuracy and reliability of farmer payments.